

Invariably Abandoned

React Single Page Applications & Docker: An unlikely match



DanFromDownUnder

04 Feb 2026 • 5 min read

Ready to go

You've created your perfect single-page application, you have your bundler all configured (whether that be vite, create-react-app or some new kid on the block) and you're ready to make it available to the world.

Should you deploy to your favourite object storage (s3, r2, etc), and put your favourite CDN proxy in front of it (Cloudflare, CloudFront etc)?

Or should you consider deploying your SPA with docker?

Usually, the answer is: go ahead with the object storage and proxy and call it a day. This is almost definitely the cheaper option for a small project (free or near-free), will perform great, and will not require you to look after any infrastructure. This approach is also widely documented and there are cookbooks, baked ci jobs and blog posts about it in abundance.

But, there are a few cases where Docker may be the better option for you...

Why Docker

Some reasons you might consider Docker:

- You want your entire stack (client, API, other resources) to be consolidated in a helm template or docker-compose file.
- You want to deploy the same build artifact to different environments (maybe staging, ci, and prod) but vary environment variables (traditional SPA tools like vite or create-react-app make this difficult and you're forced to build the application multiple times - one for each environment).
- You want to be able to deploy your application locally the same as you would in the cloud/production without the inconvenience and complexity that comes with using something like localstack.

First Ideas

Two simple approaches come to mind when thinking about how to get from your SPA source into a ready-for-prod docker container.

Static Container

The first approach would be to build your application either outside the container or using multi-step builds and copy it straight into a proven static web server container (like nginx).

Here's an example `Dockerfile` :

```
FROM node:16 as build

ARG VITE_WELCOME_MESSAGE

COPY . .
RUN npm ci
RUN npm run build

FROM nginx:1.22.1

COPY --from=build dist /usr/share/nginx/html
```

// TODO talk about the problem with routing

You have already spotted a potential problem with this... //TODO talk about env variables

For some applications, this can be perfect, but as soon as you have any environment variables (maybe an API URL, a google analytics id or anything else you need to configure) the approach falls over as vite, create-react-app and others need their environment variables at **build time**, not at **run time**.

Build on Container Start

What about deploying our source into the docker container and having it built as soon as it starts? Here's an example `Dockerfile` :

```
FROM node:16 as node

# Copy everything including our source, package.json and node_modules
COPY . .

# Install the serve npm package
RUN npm i -g serve

# On container start, build the app and serve it
CMD <<EOR
  npm run build
  npx serve ./build
EOR
```

This solution works 🎉 - and is super simple! Unfortunately, it comes with a downside which is usually a deal-breaker: performance.

Modern tooling tends to use a lot of resources to build a production bundle, in the best case, this means your application takes a very long time to become available - sometimes minutes depending on the hardware (a pain you will feel every time you push an update). And in the worst case where you have a Kubernetes cluster with resource limits, it will result in your container being evicted for being too hungry.

The other problem with this approach is we now have a much larger image size than we need because the full application source and dev dependencies

`node_modules` are included in our docker container.

Best of both worlds

Maybe we can have our cake and eat it too. Let's take our initial static container idea and modify it to allow us to still inject configuration at runtime.

Instead of using the built-in environment variable support of our toolchain (in `create-react-app` it's `process.env` and using `vite`, it's `import.meta.env`) let's roll our own.

Config file generation

First, we need to take our configuration (provided either by a `.env` file or by environment variables) and generate a js file that our application can consume.

// TODO describe the implementation

Config file consumption

Next, we need to include our new file as a script and update each place that was formerly using the toolchain config method to simply read it from our new window variable.

// TODO describe the implementation

Config validation

Since we don't know the configuration at build time anymore, we've introduced a new risk, what if we're expecting a config value to be a string and it's a number? What if a config value is completely missing? Since we're so late in the process this can't be caught with the compiler, with unit tests and not even with manual testing (as it could be that our staging environment is set up with the correct variables but production is not).

Additionally, it would be easy depending on where we're running our container to include sensitive information in environment variables that we don't want to include in our env file and make available to users of our application.

Running our environment variables through some validation can guard against both of these cases...

```
// TODO describe the implementation
```

Now our `window.env.js` includes only the values we've defined in our schema and if we're either missing a config value or it doesn't match what we're expecting we can exit with a non-zero code causing the container to shut down (and hopefully for our infrastructure to keep the old container live).

Config cache invalidation

The astute among you may have had red flags raised when we included a simple import of `window.env.js` in the index file. What about when we push an update? How will the browser know it needs to get a new version of the env file and not just used its cached value?

Let's implement some basic cache invalidation based on a hash of the config values...

```
// TODO describe the implementation
```

```
// TODO also check out etags here
```

Final Docker container

We now need to build a docker image capable of not just serving our resulting static files, but also of running our environment variable validation and generation.

```
// TODO describe the implementation
```

docker-react

I deployed the above approach enough times to be sick of repeating myself and so have gone ahead and started a [GitHub project](#) which includes a docker image you can use for this, as well as an npm package to perform the main actions for you.

Blog Post Repository

All code examples used in this article are available in full in the GitHub repository. Please also use the issue tracking to submit any feedback or corrections for this article.

GitHub - invariablyabandoned/react-spa-docker: Code demos accompanying blog article: React Single Page Applications & Docker: An unlikely match
Code demos accompanying blog article: React Single Page Applications & Docker: An unlikely match - GitHub -...

 GitHub • invariablyabandoned

blyabandoned/
spa-docker

accompanying blog article: React Applications & Docker: An unlikely

 0
Issues

 0
Stars

 0
Forks

Further Reading

- If you're interested in looking at someone else's implementation of a similar thing: Since writing this I have found [this one by coding with Manny covering Gatsby](#).

LED Status Panel

I recently completed adding slightly over 10m of LED strips to act as ambient and party lights in one of the rooms of our apartment. I was left with somewhere around 1m of SK6812 strip and thought about some good use for them. I came up with the idea of

04 Feb 2026 1 min read

Invariably Abandoned © 2026

[Sign up](#)

Powered by Ghost